

Experiment No.	4
Experiment Title	Classification using Linear and Kernel-Based Models
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02.08.2026

1 Objective

The goal of this experiment is to build a spam classifier using two different types of models, Logistic Regression and Support Vector Machine, and to see how each one behaves on the same dataset. Along with training the models, the experiment also looks at how tuning the hyperparameters of each algorithm changes the classification results, so that the two approaches can be compared on a fair basis rather than just with their default settings.

2 Problem Statement

Given a set of numerical features extracted from an email (word frequencies, character frequencies and statistics about capital letters), the task is to predict whether the email is spam or ham (not spam). This is a supervised binary classification problem: the input is a 57-dimensional feature vector per email and the output is a single label, 1 for spam and 0 for ham. The objective is to train both a linear model (Logistic Regression) and a kernel-based model (SVM), tune their hyperparameters, and evaluate which one generalizes better to unseen emails.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository / Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4601
Number of Features	57 (54 word/character frequency features + 3 capital-run-length features)
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	None (0 missing entries found across all columns)
Train-Test Split	80% train / 20% test, stratified on the class label (3680 train, 921 test samples)

4 Exploratory Data Analysis

Before training any model, the dataset was inspected to understand how the classes are distributed and how the features relate to each other and to the target.

Dataset Overview and Statistical Summary

All 57 columns are numeric (frequency or count style values), so no categorical encoding is required. A `describe()` call on the dataframe shows that most of the word-frequency columns are heavily right-skewed and dominated by zeros, since a given word usually does not appear in a given email, while a small number of high-frequency emails pull the mean upward. This skew is one of the reasons feature scaling matters before training a distance/margin based model such as SVM.

Missing Value Analysis

Running `df.isnull().sum()` on every column returns zero missing values throughout the dataset, so no imputation was needed.

Class Distribution

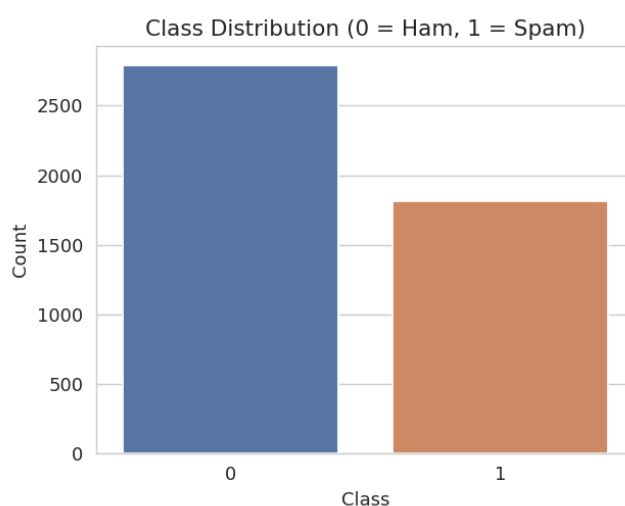


Figure 1: Distribution of spam (1) and ham (0) emails in the Spambase dataset.

Inference: Out of 4601 emails, 2788 are ham and 1813 are spam, roughly a 61:39 split. This is a mild class imbalance, not severe enough to require resampling techniques such as SMOTE, but it is still a good reason to look at precision, recall and F1-score alongside accuracy rather than accuracy alone. Stratified splitting was used for exactly this reason, so both the training and test sets keep the same class ratio.

Feature Distribution

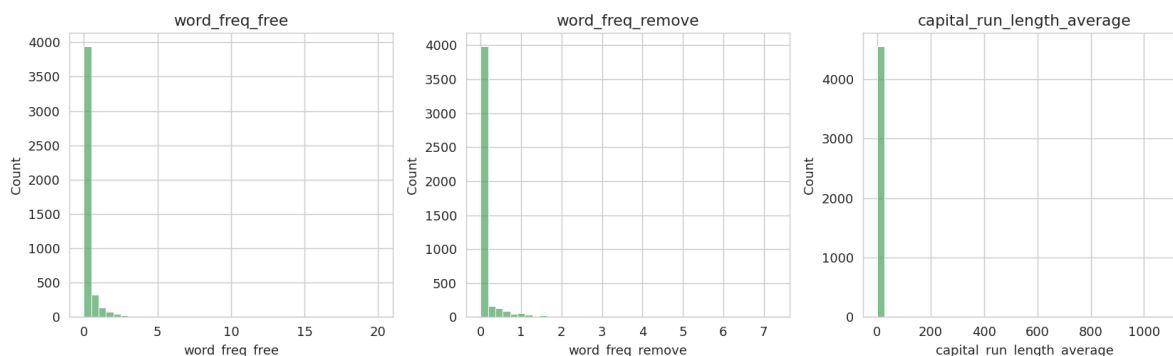


Figure 2: Distribution of three representative features: `word_freq_free`, `word_freq_remove`, and `capital_run_length_average`.

Inference: All three distributions are heavily skewed to the right, with most emails having a value at or close to zero and a long tail of high-frequency outliers. This confirms that the raw features are on very different scales (word frequencies are small fractions while capital-run-length values can run into the hundreds), which is exactly the situation where standardizing the features before training becomes important, especially for SVM, since its margin calculation is sensitive to feature scale.

5 Data Preprocessing

Since the Spambase dataset is already fully numeric and has no missing entries, preprocessing here is fairly light:

- **Missing value handling:** Not required, as confirmed in the EDA step, the dataset has zero missing values.
- **Encoding:** Not required, since every feature is already a numeric frequency or count and the target is already 0/1.
- **Feature scaling:** All 57 features were standardized using `StandardScaler` (zero mean, unit variance), fit only on the training set and then applied to the test set to avoid data leakage. This step is essential for Logistic Regression convergence and for SVM, since both are sensitive to the scale of the input features.
- **Train-test split:** An 80/20 stratified split (`random_state=42`) was used so that the class ratio in Section 3 is preserved in both subsets.
- **Feature engineering:** No new features were engineered; the original 57 Spambase features were used as-is, since they are already fairly informative for this task.

6 Machine Learning Algorithm

Logistic Regression

Logistic Regression is a linear model for binary classification. It computes a weighted sum of the input features and passes it through the sigmoid function to obtain a probability:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

A threshold of 0.5 is then applied to this probability to assign a class label. The model is trained by minimizing the log-loss (cross-entropy) between the predicted probabilities and the true labels, typically using gradient-based solvers. Regularization (L1 or L2) is added to this loss to control overfitting; L1 tends to push some coefficients to exactly zero (useful for feature selection), while L2 shrinks all coefficients smoothly.

Advantages: Fast to train, easy to interpret since each feature has a single coefficient, and works well when the classes are close to linearly separable, which is roughly true for Spambase.

Limitations: Struggles with problems where the decision boundary is genuinely non-linear, and can be sensitive to unscaled or highly correlated features.

Support Vector Machine (SVM)

SVM tries to find the hyperplane that separates the two classes with the maximum possible margin. Only the points closest to the boundary, the support vectors, determine where this hyperplane lies. For data that isn't linearly separable, SVM uses the kernel trick to implicitly map the data into a higher-dimensional space where a linear boundary can still be found. This experiment compares four kernels: linear, polynomial, RBF and sigmoid. The RBF kernel in particular is controlled by a γ parameter that decides how far the influence of a single training point reaches.

Advantages: Effective in high-dimensional spaces (57 features here), and the kernel trick lets it capture non-linear patterns that Logistic Regression cannot.

Limitations: Slower to train and tune on larger datasets, harder to interpret than Logistic Regression, and performance depends heavily on the choice of kernel and its hyperparameters.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Windows 11 / Ubuntu 22.04
Hardware	Intel Core i5, 8 GB RAM

8 Hyperparameter Selection

Grid Search (5-fold CV) was used for Logistic Regression over {C, penalty, solver}, and Randomized Search (5-fold CV, 10 iterations) was used for SVM over {kernel, C, gamma,

degree}, since the SVM search space is larger and a full grid search across four kernels would be considerably slower for a modest accuracy gain.

Parameter	Value
<i>Logistic Regression (best from Grid Search)</i>	
C	1
Penalty	L2
Solver	saga
Best CV Accuracy	0.9242
<i>SVM (best from Randomized Search)</i>	
Kernel	RBF
C	1
Gamma	scale
Best CV Accuracy	0.9312

9 Results

Metric	Logistic Regression	SVM (Tuned)
Accuracy	0.9305	0.9273
Precision	0.9211	0.9277
Recall	0.9008	0.8843
F1-score	0.9109	0.9055
ROC-AUC	0.9704	0.9667

10 Result Visualization

SVM Kernel Comparison

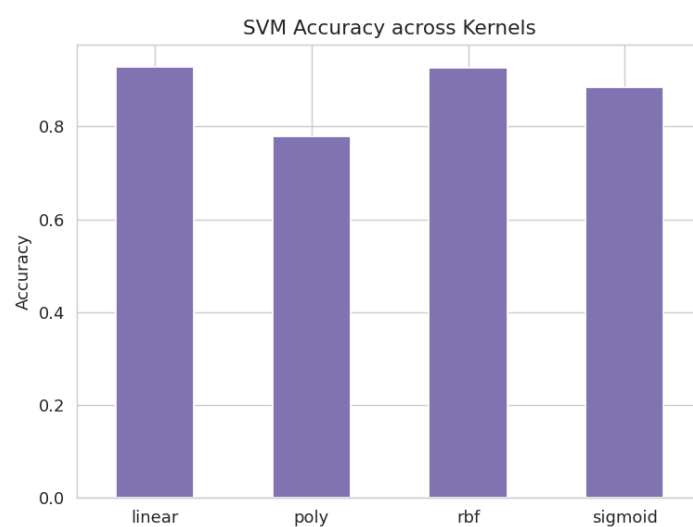


Figure 3: Test accuracy of SVM with linear, polynomial, RBF and sigmoid kernels (default hyperparameters).

Inference: The linear and RBF kernels perform almost identically (around 0.93 accuracy) and clearly outperform the sigmoid kernel, while the polynomial kernel (degree 3, default settings) does noticeably worse, dropping to about 0.78. This suggests the true decision boundary for spam detection is close to linear, so a plain polynomial kernel without tuning actually distorts the boundary rather than helping. It also explains why the hyperparameter search converged on an RBF kernel with a low-degree/near-linear behaviour rather than a high-degree polynomial.

Confusion Matrices

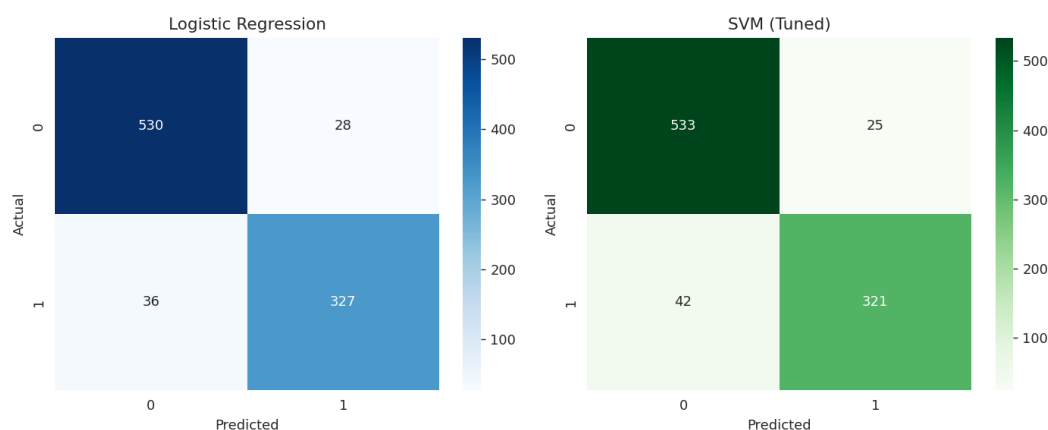


Figure 4: Confusion matrices for the tuned Logistic Regression (left) and tuned SVM (right) on the test set.

Inference: Both models make a similar number of errors overall, but the pattern differs slightly. Logistic Regression catches marginally more actual spam emails (higher recall), while SVM makes fewer false alarms on genuine ham emails (higher precision). For a spam filter, false positives (ham marked as spam) are usually more costly than false negatives, so SVM's slightly higher precision could be preferable in a real deployment even though its overall accuracy is a touch lower.

ROC Curve

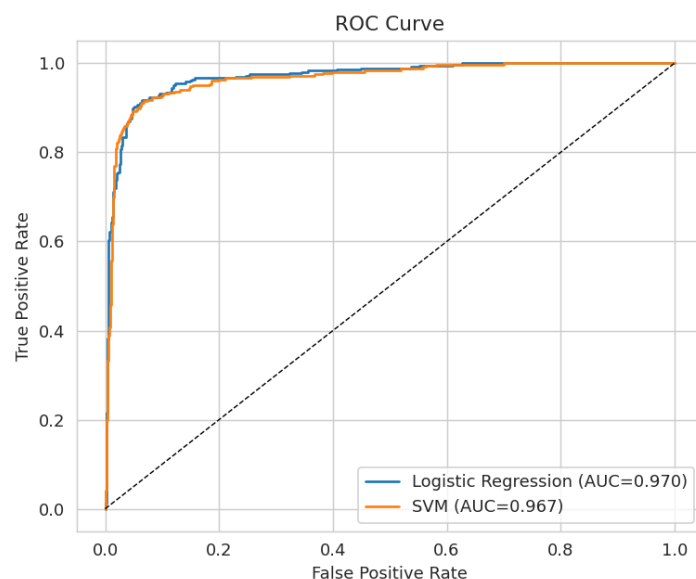


Figure 5: ROC curves for Logistic Regression and SVM on the test set.

Inference: Both curves stay well above the diagonal (random-guess) line, and the AUC values are close, 0.970 for Logistic Regression versus 0.967 for SVM. This tells us both models rank spam emails above ham emails correctly for the large majority of threshold choices, and the slightly higher AUC for Logistic Regression suggests it is marginally better at separating the two classes across all possible thresholds, not just the default 0.5 cutoff.

Precision-Recall Curve

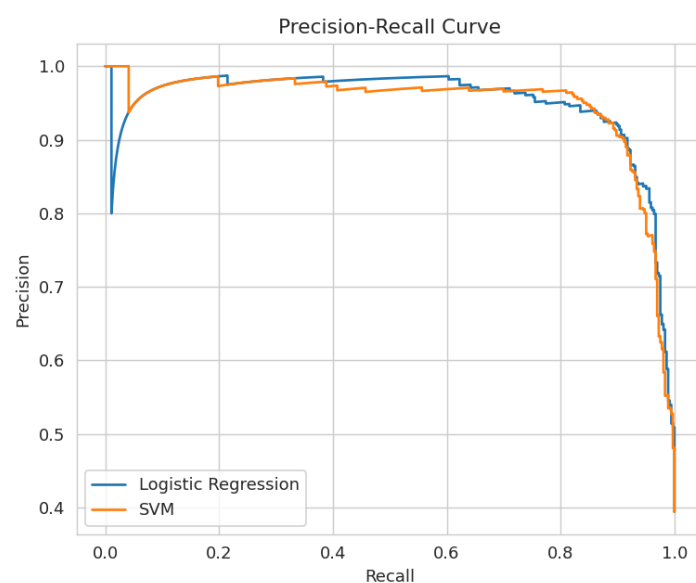


Figure 6: Precision-Recall curves for Logistic Regression and SVM on the test set.

Inference: Given the moderate class imbalance (61:39), the precision-recall curve is arguably more informative here than the ROC curve. Both curves stay high across most of the recall range, dropping off only near recall values close to 1, which means both models can be pushed to catch nearly all spam emails while still keeping precision reasonably high, though the exact trade-off point would need to be chosen based on how costly false positives are for the intended user.

Learning Curve

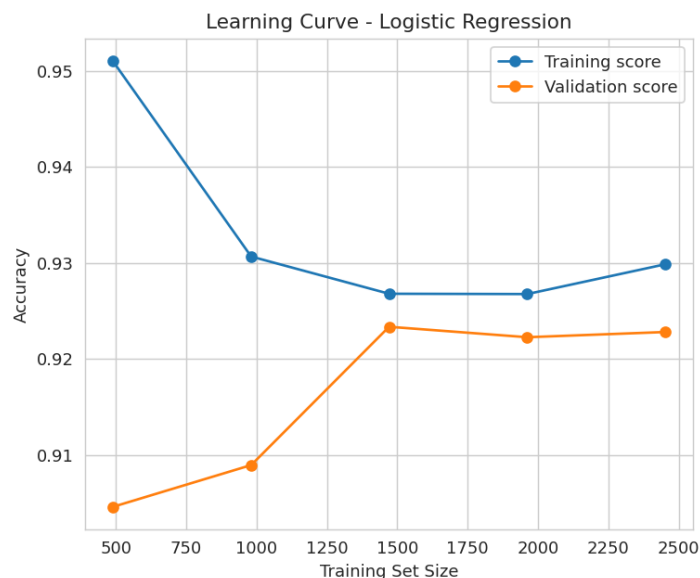


Figure 7: Learning curve for the tuned Logistic Regression model.

Inference: The training and validation accuracy curves converge and stay close together as the training set grows, with only a small gap between them. This indicates the model is not overfitting significantly, and that adding more training data beyond what is already available would likely give only marginal further improvement, since the validation score has largely plateaued.

Feature Importance



Figure 8: Top 15 Logistic Regression coefficients by absolute value.

Inference: Features like `char_freq_%21` (the ! character), `word_freq_remove`, and `word_freq_free` have the largest positive coefficients, meaning a higher frequency of these strongly pushes the prediction toward spam. A few features such as `word_freq_george` and `word_freq_hp` have negative coefficients, pulling predictions toward ham; this makes sense since these were common terms in the personal/business mailbox the original dataset was collected from, so their presence is actually a hint that the email is legitimate.

11 Discussion

Both models end up performing quite similarly on this dataset, with Logistic Regression slightly ahead on accuracy, recall and ROC-AUC, and SVM slightly ahead on precision. The small gap between them (roughly 0.3 to 0.5 percentage points on most metrics) suggests the Spambase classes are close to linearly separable, which favours a linear model that trains in a couple of seconds over an RBF-kernel SVM that took over ten times longer to fit on this dataset. Regularization played a visible role for Logistic Regression: the grid search picked a moderate C value (1) with L2 penalty, avoiding both the underfitting seen at very small C and the mild overfitting risk at large C. For SVM, the kernel comparison in Section 8 makes it clear that an untuned polynomial kernel performs poorly here, while linear and RBF kernels with reasonable hyperparameters are essentially interchangeable in accuracy, confirming a low-variance, low-bias sweet spot for this particular dataset. One limitation of this study is that the SVM search used Randomized Search with only 10 iterations for time reasons, so a full grid search might squeeze out a slightly better configuration, though based on how close linear and RBF already are, any improvement is unlikely to be large. Another limitation is that no explicit handling of the 61:39 class

imbalance was done beyond stratified splitting; techniques like class weighting could be explored to push recall higher without hurting precision much.

12 Conclusion

This experiment trained and tuned Logistic Regression and SVM classifiers to detect spam emails on the Spambase dataset. After hyperparameter tuning, Logistic Regression reached 93.05% test accuracy and an F1-score of 0.911, while SVM (RBF kernel) reached 92.73% accuracy and an F1-score of 0.906, essentially matching each other despite Logistic Regression being a far simpler and faster model. The confusion matrix and precision-recall analysis suggest that in a real spam-filtering application the choice between the two would come down to whether false positives (ham marked as spam) or false negatives (spam reaching the inbox) matter more, since SVM edges out on precision while Logistic Regression edges out on recall. Overall, the experiment shows that a well-tuned linear model can be competitive with, and in this case cheaper than, a kernel-based model when the underlying classes are close to linearly separable.

13 References

- [1] Scikit-learn developers, “Logistic Regression,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- [2] Scikit-learn developers, “Support Vector Machines,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
- [3] Somesh24, “Spambase Dataset,” *Kaggle*. [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>